PIC Bug Tracker

The tracker is **GitHub Issues** on the project repository. This page explains how to report, rank, fix and close a bug so problems get fixed fast and nothing is lost.

Contents

1. Where things live
2. Reporting a bug
3. Severity
4. Life of an issue
5. Labels
6. Fixing a bug
7. Weekly routine
8. One-time setup
9. First issues

1. Where things live

What	Where
Issues	GitHub repo → **Issues**
Issue forms	Bug, Feature or missing workflow, Security (`.github/ISSUE_TEMPLATE/`)
Labels	`.github/labels.yml` (a workflow keeps GitHub in step with it)
Pull request checklist	`.github/pull_request_template.md`
Board	GitHub repo → **Projects** (one board, set up in §8)
Open workflows from the specs	PIC_Pending_Workflows.md
Runbooks for live incidents	`docs/runbooks/` (dead-lettered postings, workflow incidents, MRP failure, secret rotation, incident response)

2. Reporting a bug

1. GitHub → **Issues** → **New issue** → **Bug report**.
2. Fill in: the area, the severity, where you saw it, the company and user, the document number, the steps, what you expected (name the flow step or spec section if you know it, for example F7-04) and what happened.
3. **Include the** `traceId` from the error message when there is one. It finds the request in the logs and the audit trail.
4. **Never paste** passwords, tokens, connection strings or real bank and tax IDs.
5. An exploitable security hole is **not** reported in a public issue. Use the private link on the new-issue page.

3. Severity

Level	Meaning	Examples	Target
S1 Critical	Wrong money or data, a security hole, or the system is down	A supplier paid twice; one company sees another's data; sign-in fails for everyone	Start now; fix or roll back the same day
S2 High	A workflow is blocked or a control is bypassed, with no workaround	An approval chain skips a required approver; invoices cannot post	Fix within 3 working days
S3 Medium	Wrong behaviour that has a workaround	A filter returns the wrong rows; a report total is off	Fix within 2 weeks
S4 Low	Cosmetic or minor	A wrong label, a misaligned column	Next convenient release

If unsure, pick the higher level; triage will adjust it.

4. Life of an issue

Step	Label	Who	What happens
1	`status: triage`	Reporter	Issue is filed from a form
2	`status: ready`	Triage (daily)	Reproduced, severity confirmed, area and owner set
3	`status: in progress`	Owner	Branch created; the issue is linked in the pull request
4	closed	Reviewer	Merged pull request with `Fixes #123`; the issue closes by itself
—	`status: blocked`	Owner	Waiting on someone; the issue says who and for what
—	`status: needs client`	Triage	Needs the client's system, data or decision

Triage means: reproduce it, check it is not a duplicate, set the severity and one `area:` label, and name an owner.

5. Labels

- **Type:** `type: bug`, `type: feature`, `type: security`, `type: debt`
- **Severity:** `severity: S1` to `severity: S4`
- **Status:** `status: triage`, `ready`, `in progress`, `blocked`, `needs client`
- **Area:** one per workflow or layer: F1 planning, F2 requisitions, F3 sourcing, F4 contracts, F5 orders, F6 receiving, F7 invoice-pay, F8 suppliers, F9 master data, spend-savings, AI-agents, admin-rules, portal, platform, erp-migration, deploy-ci, docs
- **Other:** `industry`, `from: code review`, `from: spec matrix`

6. Fixing a bug

1. **Reproduce it with a failing test first.** Backend tests are in `backend/tests/pic/`; use a rolled-back transaction or your own objects so the signature-case data stays unchanged.
2. Fix the cause, not the symptom. Check the callers and any other place with the same pattern.
3. Open a pull request with `Fixes #123`. The pull request checklist covers tests, type check, lint, migrations, secrets, controls and docs.
4. **CI must be green** before merging. CI runs the backend suite, the frontend type check and lint, the dependency audits, the container build, and the security scans.
5. A database change needs a migration with upgrade and downgrade tested, and the existing data checked against any new constraint.
6. After merge, a push to `main` runs the Neon workflow, which applies migrations and verifies the database.

For a live S1: stop the damage first (block the supplier, switch the rule off, roll back the release per `docs/runbooks/release-and-rollback.md`), then fix properly.

7. Weekly routine

When	Who	What
Daily	Triage owner	Clear `status: triage`; confirm S1 and S2 have an owner
Weekly	Team	Review every S1/S2 older than its target; review `status: blocked` and `needs client`
Before each release	Release owner	No open S1; S2s each have a decision (fix or accept in writing)
Monthly	Project lead	Count opened, closed and average age by severity; look for areas with repeat bugs

8. One-time setup

1. **Labels:** push this change to `main`. The "Sync labels" workflow creates them. You can also run it by hand under **Actions** → **Sync labels** → **Run workflow**.
2. **Board:** GitHub repo → **Projects** → **New project** → **Board**. Add the columns *Triage, Ready, In progress, Blocked, Done*. In the project's **Workflows**, switch on "Item added to project → Triage" and "Item closed → Done", and **Add item from repository** → **Auto-add** with the filter `is:issue`.
3. **Notifications:** under **Settings** → **Notifications**, watch the repository for issues so S1 reports reach you.
4. **First issues:** see §9.

9. First issues

The tracker starts with **68 issues**: the 59 open workflows from PIC_Pending_Workflows.md and 9 known defects.

1. Create a token: GitHub → **Settings** → **Developer settings** → **Personal access tokens** → **Fine-grained tokens** → **Generate**. Select only this repository, and set **Issues: Read and write**. Copy it once.
2. In PowerShell, from the project folder (the token stays in that window only):

   ```
   $env:GITHUB_TOKEN = "paste-the-token-here"
   & "C:\Users\SSINFOTECH\Desktop\supplier-capability-twin\backend\venv\Scripts\python.exe"
   .github\scripts\seed_issues.py --create
   ```

3. It prints `created 68, skipped 0`. Running it again creates nothing new.
4. Delete the token on GitHub when done.

The script without `--create` only lists what it would create.